MILITARY INSTITUTE OF SCIENCE & TECHNOLOGY

Department of Computer Science and Engineering

WALKTHROUGH

CSE 307: Operating Systems (Part B)

Decision Tree Based Page Replacement Under Changing Memory Access Patterns

Submitted By

Fahim Fazle Aziz Bin Zahid

ID: 202414004

Level 3, Term 1, Section A

GitHub Repository

https://github.com/Fahim5492/LearnedAlgoDecisionTree.git

Submitted To

Lec Khaled Hasan Irfan

Lecturer

Department of Computer Science and Engineering

Military Institute of Science & Technology

1. Purpose of This Walkthrough

This document provides a complete technical walkthrough of the project Decision Tree Based Page Replacement Under Changing Memory Access Patterns.

The project was originally intended to include an in-class code walkthrough. Since the live demonstration could not be conducted, this document serves as the written replacement for that walkthrough.

The walkthrough explains:

- the development environment and repository structure,
- the responsibility of each Python source file and its important functions,
- how synthetic workloads and supervised-learning examples are generated,
- how the Decision Tree is trained, validated, and used by the learned replacement policy,
- how the workload-shift experiment and multi-seed evaluation are performed,
- how confidence and Optimal-victim agreement are analyzed,
- how Belady's anomaly and arbitrary manual traces are tested,
- and how the complete project can be reproduced from the command line.

The term paper contains the theoretical discussion and interpretation of the results. This walkthrough focuses on implementation, code structure, execution flow, and reproducibility.

2. Development Environment

The project was developed and tested inside a Kali Linux virtual machine running through Oracle VirtualBox.

The main development environment was:

- Kali Linux
- Oracle VirtualBox
- Python 3
- Python virtual environment (venv)
- scikit-learn
- joblib
- matplotlib
- Git

Kali Linux was the development and testing environment, but the Python implementation itself does not depend on Kali-specific operating-system features.

A Python virtual environment was used to keep project dependencies separate from the system Python installation.

```
python3 -m venv .venv

source .venv/bin/activate

python3 -m pip install scikit-learn joblib matplotlib
```

Fixed random seeds are used for reproducibility. The workload parameters, recent-frequency window, frame/page configuration, and Decision Tree depth are fixed design values for this controlled experiment; they were not selected through parameter tuning.

3. Project Directory Structure

```
LearnedAlgoDecisionTree/
├── src/
│   ├── fifo.py
│   ├── lru.py
│   ├── optimal.py
│   ├── learned.py
│   ├── workload.py
```

```
│   ├── generate_dataset.py
│   ├── train_model.py
│   ├── save_model.py
│   ├── experiment.py
│   ├── evaluate.py
│   ├── confidence_analysis.py
│   ├── confidence_summary.py
│   ├── demo.py
│   ├── belady_test.py
│   └── plot_results.py
├── data/
│   └── training_data.csv
├── models/
│   └── decision_tree.joblib
├── results/
│   ├── results.csv
│   ├── confidence_results.csv
│   ├── confidence_summary.csv
│   ├── summary_table.csv
│   ├── hit_ratio_comparison.png
│   ├── page_fault_comparison.png
│   └── confidence_summary_chart.png
├── report/
│   ├── Term_Paper.pdf
│   └── Walkthrough.pdf
├── .gitignore
└── README.md
```

Folder	Purpose
src/	Python implementation files.
data/	Generated Decision Tree training dataset.
models/	Serialized final Decision Tree.
results/	Experimental CSV files, summaries, and figures.
report/	Final term paper and walkthrough document.

4. Overall System Flow

```
Synthetic workload generation
            ↓
Optimal-based candidate labeling
            ↓
training_data.csv
            ↓
Decision Tree construction
            ↓
Learned replacement policy
            ↓
Workload-shift experiment
            ↓
Multi-seed evaluation
            ↓
Confidence analysis
            ↓
Result summaries and figures
```

The training dataset is generated deterministically. The dataset-generation entry point uses a fixed seed, and the individual trace seeds are derived from that seeded random-number generator. Regenerating the dataset with the same code therefore reproduces the same training traces and candidate examples.

The model used by the current experiment path is rebuilt from training_data.csv through build_learned_model(). The Decision Tree also uses a fixed random_state=42, so building the model from the same dataset is deterministic under the same software environment.

A serialized model is created separately for persistence:

```
training_data.csv
        ↓
save_model.py
        ↓
models/decision_tree.joblib
```

The Joblib file is a stored copy of the final trained model. The current experiment scripts do not require it.

5. Classical Page-Replacement Files

5.1 fifo.py

fifo_page_replacement(reference_string, frame_count)

Implements FIFO page replacement. The function receives a page-reference sequence and frame count, maintains the current frames, hit/fault counters, and a step-by-step history, and evicts the page that entered memory first whenever replacement is required.

It returns:

```
hits
faults
hit_ratio
history
```

The history is later reused to calculate phase-specific results without restarting the simulation.

5.2 lru.py

lru_page_replacement(reference_string, frame_count)

Implements LRU replacement. Resident pages are maintained in recency order. A hit moves the page to the most-recent position, while a replacement removes the least-recently-used page.

The function returns the same hits, faults, hit_ratio, and history structure as FIFO so that later scripts can process policies consistently.

5.3 optimal.py

optimal_page_replacement(reference_string, frame_count)

Implements the future-aware Optimal benchmark. On a replacement, the function inspects future references. A page that never appears again is immediately suitable for eviction; otherwise the candidate whose next use is furthest in the future is selected.

Optimal serves two roles: it is the theoretical comparison benchmark and the teacher used to create supervised training labels.

6. Workload Generation

6.1 workload.py

This file generates all synthetic page-reference traces. It contains four general workload families for training and one locality-to-random generator for the final evaluation.

generate_locality_trace(...)

Creates a locality-heavy trace. A small working set is chosen from the virtual pages. With the configured locality probability, the next reference is selected from that working set; otherwise it is selected from outside the working set.

The final experiment uses a working-set size of 4 and locality probability of 0.80. This creates strong but imperfect locality: most references remain inside the working set while some still leave it.

generate_random_trace(...)

Creates uniformly random references over the complete virtual-page range. There is no persistent hot working set, which makes historical recency and recent frequency much less informative.

generate_sequential_trace(...)

Creates sequential page access with wrap-around, providing a structured training pattern different from locality-heavy reuse.

generate_bursty_trace(...)

Creates temporary hot sets that change over time. Each burst selects a hot set and burst length, produces most references from that set, occasionally references other pages, and then moves to a new hot set.

generate_training_traces(...)

Creates 20 broad training traces by generating five traces from each workload family: locality, sequential, random, and bursty. Each trace contains 2,000 references.

The collection is balanced at the trace-count level. Candidate-row counts can still differ by workload family because different workloads produce different numbers of page faults. The trace list is shuffled before being returned.

generate_shift_trace(...)

Creates the final evaluation trace by concatenating a locality-heavy phase and a uniform-random phase. The standard configuration uses 20 virtual pages, 4 physical frames, a four-page locality working set, locality probability 0.80, and 1,000 references per phase.

Using 20 virtual pages with only 4 frames creates regular replacement pressure while keeping the eviction candidate set small enough to inspect clearly. With one recorded Optimal target among four candidates, uniform random victim selection has a 25% exact-agreement baseline.

7. Training Dataset Generation

7.1 generate_dataset.py

This file converts generated traces into supervised-learning candidate rows. Each row represents one resident page that could potentially be evicted.

choose_optimal_victim(frames, reference_string, current_step)

Determines which candidate page Optimal would remove. If a resident page never appears again, the function returns it immediately. Otherwise, it selects the page whose next use occurs furthest in the future.

If several resident pages never appear again, they are equally valid under the future-use criterion. The implementation returns the first such page encountered in the current frame order. This is why later confidence analysis is described as agreement with the recorded Optimal victim rather than a complete measure of replacement quality.

recent_frequency(reference_string, current_step, page, window_size)

Counts how many times a candidate page appeared during a recent historical window. The project uses a 50-reference window so that the feature measures recent reuse instead of accumulating frequency over the entire trace.

extract_training_examples(...)

Processes one complete trace while following Optimal replacement. Whenever a page fault occurs with all frames occupied, every resident page becomes a candidate. Recency and recent frequency are calculated for each candidate, the recorded Optimal victim receives label 1, and all other candidates receive label 0.

Each row also records trace_id, workload_type, step, and candidate_page. After labeling the candidates, the simulation follows the recorded Optimal victim and continues.

generate_dataset(...)

Calls the training-trace generator, extracts examples from every trace, combines them, and writes data/training_data.csv. The resulting dataset contains approximately 75,340 candidate examples.

The normal dataset-generation path begins with a fixed seed of 100, and all derived trace seeds are generated deterministically from it.

8. Decision Tree Training and Validation

8.1 train_model.py

This file performs model training and held-out-trace validation. It does not supply the model object used by the experiment scripts.

load_dataset(file_path)

Reads data/training_data.csv and converts the stored CSV values into Python values suitable for model training.

split_by_trace(rows, seed=42)

Holds out one complete trace from each of the four workload families. Therefore, 4 of the 20 generated traces are used for validation and the remaining 16 provide training examples.

All candidate rows from a held-out trace remain outside the training set, preventing examples from the same page-reference trace from appearing on both sides of validation.

prepare_features(rows)

Converts candidate rows into X = [recency, recent_frequency] and y = label, where 0 means non-victim and 1 means the candidate selected by Optimal.

train_decision_tree(x_train, y_train)

```
DecisionTreeClassifier(
    max_depth=5,
    class_weight="balanced",
    random_state=42
)
```

Depth 5 keeps the tree relatively small and interpretable while still allowing nonlinear combinations of the two features. Balanced class weighting compensates for the candidate-level imbalance created by one positive victim and several negative candidates per replacement.

Validation output

Running python3 src/train_model.py prints training and validation counts, held-out trace IDs, a classification report, candidate-classification accuracy, feature importances, and the learned tree rules.

The Decision Tree trained on the 16 training traces and evaluated on the four held-out traces achieved 82.29% candidate-classification accuracy. A trivial classifier that labels every candidate as a non-victim would obtain about 75% accuracy, so the validation result is 7.29 percentage points above that baseline.

For this validation tree, the reported feature importances were approximately 79.84% for recency and 20.16% for recent frequency.

> *Important: the 82.29% accuracy and 79.84% / 20.16% feature importances describe the validation tree trained on 16 traces. The experimental model described in Section 10 is trained again using all 20 traces.*

9. Model Serialization

9.1 save_model.py

This file trains a final Decision Tree using the complete generated dataset and stores it with Joblib.

```
python3 src/save_model.py
```

Output:

```
models/decision_tree.joblib
```

The serialized file preserves a reusable copy of the final full-data model. It is not required by the current experiment path, which rebuilds the model directly from training_data.csv.

10. Learned Replacement Policy

10.1 learned.py

build_learned_model(...)

Loads all rows from the generated training dataset and trains the final Decision Tree used by the learned replacement policy. Unlike train_model.py, it does not hold out validation traces. It uses all 20 generated traces because validation has already been performed separately.

This means the model used in the experiments is not exactly the same tree as the validation tree whose 82.29% accuracy and feature importances were reported in Section 8.

learned_page_replacement(reference_string, frame_count, model, window_size=50)

Runs the learned page-replacement simulation. It maintains resident frames, last-access positions, hit/fault counters, and detailed history.

On a full-frame fault, every resident page becomes a candidate. Recency and recent frequency are calculated, model.predict_proba(...) provides an eviction-class score, and the candidate with the highest score is removed. Recency is used as the tie-breaker when candidate scores are equal.

For every learned eviction, the history additionally stores frames_before, the learned victim, confidence, and all candidate scores. These values are required for decision-level confidence analysis.

11. Single Workload-Shift Experiment

11.1 experiment.py

Runs one detailed instance of the final locality-to-random workload shift.

calculate_phase_metrics(history, start_step, end_step)

Extracts a selected interval from a policy history and calculates hits, faults, and hit ratio for that phase. Because one continuous history is used, memory is not reset at the shift point.

print_policy_results(...)

Prints locality-phase, random-phase, and overall results for one policy.

Main configuration

```
physical frames      = 4
virtual pages        = 20
locality references  = 1000
random references    = 1000
working set size     = 4
locality probability = 0.80
seed                 = 42
```

The same generated reference string is executed by FIFO, LRU, Optimal, and Learned. Run the detailed experiment with:

```
python3 src/experiment.py
```

12. Multi-Seed Evaluation

12.1 evaluate.py

Repeats the shifted experiment across several generated traces and calculates averaged results.

calculate_phase_metrics(...)

Calculates hits, faults, and hit ratio for a selected phase.

collect_policy_rows(...)

Converts one policy result into locality, random, and overall records containing seed, policy, phase, hits, faults, and hit_ratio.

run_single_seed(seed, model, frame_count=4)

Generates one locality-to-random trace for the supplied seed and runs FIFO, LRU, Optimal, and Learned on exactly the same trace.

save_results(...)

Writes the detailed evaluation records to results/results.csv.

print_averages(...)

Groups results by policy and phase and calculates average hit ratio and page faults.

The final evaluation uses seeds 42 through 51, giving ten reproducible but different shifted traces. Several traces are used so the conclusions do not depend on one randomly generated example.

Program	Role
experiment.py	One detailed locality-to-random run.
evaluate.py	Ten shifted runs (seeds 42-51) and averaged results.

```
python3 src/evaluate.py
```

13. Decision-Level Confidence Analysis

13.1 confidence_analysis.py

Measures how often the learned victim agrees with the single victim returned by Optimal from the same resident frame state.

analyze_seed(seed, model, frame_count=4)

Runs the learned policy on one shifted workload. For each actual learned eviction, it retrieves frames_before, gives those exact candidates to Optimal, records both victims and the model confidence, records whether they agree, and labels the decision as locality or random phase.

```
seed
step
phase
learned_victim
optimal_victim
confidence
correct
```

The correct field means exact agreement with the single victim returned by the Optimal implementation.

Why the same candidate frames are used

Different replacement policies can diverge after making different evictions. Comparing the learned victim with a separate Optimal simulation could therefore compare different candidate sets. Passing the learned policy's actual frames_before to Optimal keeps the comparison at the same memory state.

save_results(...) and print_summary(...)

save_results(...) writes results/confidence_results.csv. print_summary(...) reports total decisions, agreements, agreement rate, average confidence, and confidence for agreeing and non-agreeing decisions across seeds 42 through 51.

```
python3 src/confidence_analysis.py
```

14. Confidence Summary

14.1 confidence_summary.py

Reduces the detailed confidence-results CSV into phase-level statistics suitable for reporting.

load_results(file_path)

Loads phase, confidence, and correct fields from results/confidence_results.csv.

summarize_phase(rows, phase)

Calculates total decisions, agreeing and non-agreeing decisions, Optimal-victim agreement, average confidence, confidence when agreement occurs, and confidence when it does not.

save_summary_csv(...)

Writes results/confidence_summary.csv.

plot_confidence_summary(...)

Creates results/confidence_summary_chart.png, comparing Optimal-Victim Agreement, Average Confidence, Confidence When Correct, and Confidence When Incorrect across the locality-heavy and random phases.

```
python3 src/confidence_summary.py
```

15. Result Plot Generation

15.1 plot_results.py

Converts the multi-seed evaluation CSV into the final performance summaries and figures.

load_results(file_path)

Reads results/results.csv.

compute_averages(rows)

Groups the data by policy and phase and calculates average hit ratio and page-fault count.

save_summary_table(...)

Creates results/summary_table.csv.

plot_hit_ratio_comparison(...)

Creates results/hit_ratio_comparison.png, comparing locality-heavy and random hit ratios for FIFO, LRU, Optimal, and Learned.

plot_page_fault_comparison(...)

Creates results/page_fault_comparison.png, comparing overall average page faults for all four policies.

```
python3 src/plot_results.py
```

16. Manual Simulation

16.1 demo.py

Allows a user to manually enter a page-reference string and frame count without changing source code.

```
python3 src/demo.py
```

The same custom input is automatically executed by FIFO, LRU, Optimal, and Learned.

print_result(name, result)

Prints hits, page faults, and hit ratio for one policy.

analyze_confidence(reference_string, learned_result)

Examines each learned eviction and records the step, requested page, learned victim, recorded Optimal victim, confidence, and agreement.

print_confidence_results(decisions)

Prints total replacement decisions, Optimal-victim agreement, confidence summaries, and individual learned replacement details.

17. Belady Anomaly Testing

17.1 belady_test.py

Tests how page-fault counts change when the number of available frames changes.

```
python3 src/belady_test.py
```

The user can select the classic Belady trace or enter a custom reference string, then select the frame counts to test.

test_policy(...)

Runs one replacement policy for each requested frame count and records hits, faults, and hit ratio. An anomaly is marked when increasing the frame count produces more page faults than the previous tested frame count.

print_policy_results(...)

Displays the frame-count comparison for FIFO, LRU, Optimal, and Learned.

Classic reference string:

```
1 2 3 4 1 2 5 1 2 3 4 5
```

For FIFO, the classic result is:

```
3 frames → 9 faults
4 frames → 10 faults
```

The learned policy did not reproduce the anomaly on this tested trace. This observation does not establish that the learned policy is anomaly-free.

18. Generated Files

File	Purpose
data/training_data.csv	Generated supervised-learning candidate dataset.
models/decision_tree.joblib	Serialized copy of the final full-data Decision Tree; stored for persistence.
results/results.csv	Detailed multi-seed performance results.
results/confidence_results.csv	Decision-level learned/Optimal agreement and confidence records.
results/summary_table.csv	Averaged performance values used by the final plots.
results/confidence_summary.csv	Phase-level agreement and confidence summary.
results/hit_ratio_comparison.png	Average hit-ratio comparison before and after the shift.
results/page_fault_comparison.png	Overall average page-fault comparison.
results/confidence_summary_chart.png	Optimal-victim agreement and confidence comparison across phases.

19. Reproducing the Complete Project

Activate the environment first:

```
source .venv/bin/activate
```

Step 1: Generate the Training Dataset

```
python3 src/generate_dataset.py
```

Output: data/training_data.csv

Step 2: Train and Validate the Decision Tree

```
python3 src/train_model.py
```

Output: Held-out-trace validation output

Step 3: Optionally Serialize the Final Model

```
python3 src/save_model.py
```

Output: models/decision_tree.joblib

> *This step is optional for reproducing the current experiments because the active experiment path rebuilds the deterministic full-data tree from training_data.csv.*

Step 4: Run One Detailed Workload-Shift Experiment

```
python3 src/experiment.py
```

Output: Console phase and overall results

Step 5: Run the Multi-Seed Evaluation

```
python3 src/evaluate.py
```

Output: results/results.csv

Step 6: Run Confidence Analysis

```
python3 src/confidence_analysis.py
```

Output: results/confidence_results.csv

Step 7: Generate the Confidence Summary

```
python3 src/confidence_summary.py
```

Output: results/confidence_summary.csv and results/confidence_summary_chart.png

Step 8: Generate the Main Result Figures

```
python3 src/plot_results.py
```

Output: results/summary_table.csv, results/hit_ratio_comparison.png, results/page_fault_comparison.png

After these steps, the complete training, validation, experiment, confidence-analysis, and visualization pipeline has been reproduced.

20. Running Only a Manual Test

A custom trace can be tested without regenerating the research dataset.

```
python3 src/demo.py
```

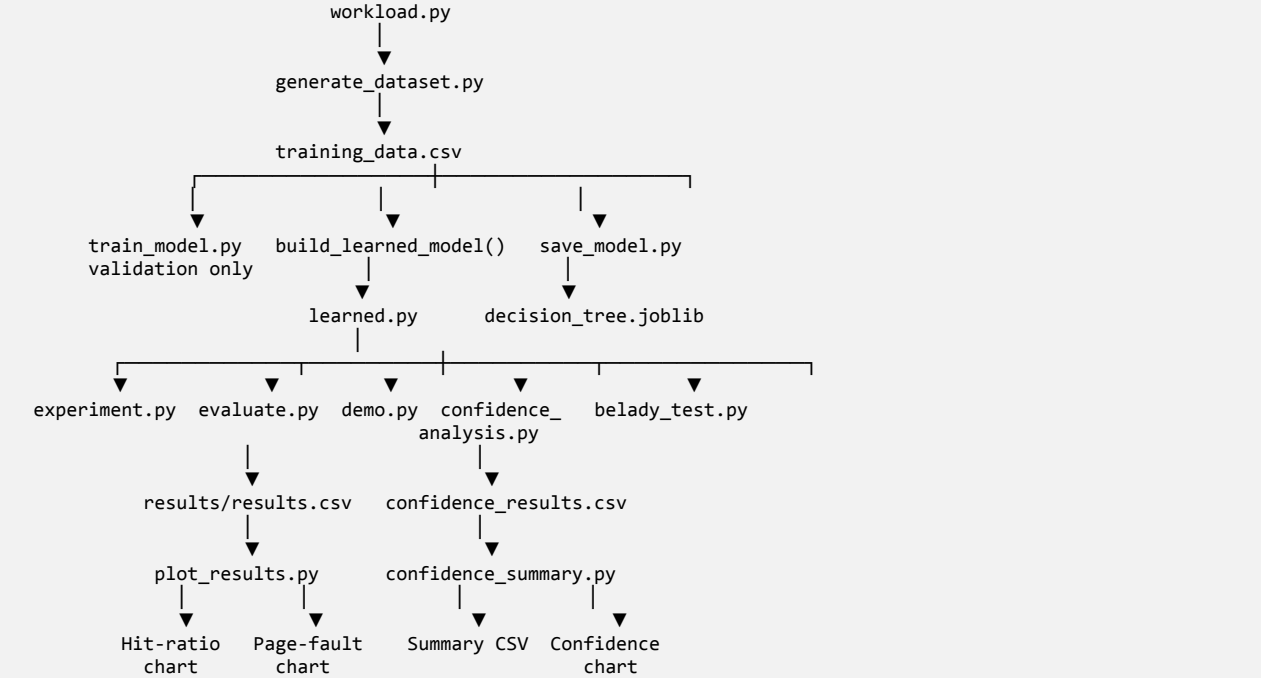
The program requests a reference string and frame count, then runs all four policies automatically.

21. Running Only the Belady Test

```
python3 src/belady_test.py
```

The program supports both the classic Belady trace and manually entered traces with user-selected frame counts.

22. Relationship Between the Main Files



train_model.py is a validation branch only. It does not feed the experimental model. Both build_learned_model() and save_model.py consume training_data.csv directly.

experiment.py and evaluate.py are independent consumers of the same workload and policy implementations; neither program feeds into the other.

Path	Purpose
training_data.csv → train_model.py	Train on 16 traces and validate on 4 held-out traces.
training_data.csv → build_learned_model()	Train the full-data model used by Learned in experiments.
training_data.csv → save_model.py	Train and serialize a full-data copy to decision_tree.joblib.
experiment.py	One detailed shifted workload.
evaluate.py	Ten shifted workloads and averaged results.
demo.py	Manual custom-trace comparison.
belady_test.py	Classic/custom frame-count anomaly testing.

23. Summary

The project is implemented as a complete experimental page-replacement pipeline rather than as a single algorithm script.

Synthetic locality-heavy, sequential, random, and bursty workloads are generated first. Optimal replacement is then used to label resident eviction candidates. The resulting dataset is used to train a Decision Tree using recency and recent frequency as historical features.

Four of the twenty generated traces are held out for trace-level validation. The reported 82.29% candidate-classification accuracy and 79.84% / 20.16% feature importances belong to this validation tree. After validation, the experimental learned model is trained again using all twenty generated traces.

The learned policy is compared with FIFO, LRU, and Optimal under a deliberate locality-to-random workload shift. experiment.py provides one detailed run, while evaluate.py repeats the experiment across seeds 42 through 51 and produces the final averaged results.

The confidence-analysis pipeline evaluates learned victims against the victim returned by Optimal from the same resident frame state. Separate summary and plotting scripts generate the CSV files and figures used in the final report.

Finally, demo.py supports manually entered workloads and belady_test.py supports classic or custom frame-count anomaly testing. Together, the source files make the project reproducible from workload generation through dataset creation, validation, learned replacement, multi-seed evaluation, confidence analysis, visualization, and manual testing.